ML Regression Models

Professional Workflow Step

Models Under Evaluation:

- **OLS** Linear Regression
- Ridge Regression (L2)
- Lasso Regression (L1)
- Decision Tree Regressor

Pipeline initialized. Proceeding with feature engineering and model training.

📈 California Housing Price Prediction

Project Goal & Methodology Overview

The primary objective is to develop a predictive model to accurately estimate **median house prices** across California districts. This task leverages the robust *California housing dataset* from `scikit-learn`, focusing on **linear regression** techniques.

🎯 **Output Target Scaling: The target variable, Median House Value (`MedHouseVal`), is presented in units of 100,000 dollars.**

To obtain the actual price, the predicted value must be multiplied by $100,000.

INPUT FEATURES

- Median income
- House age
- Rooms/Population
- Location (Lat/Lon)

🙏 Acknowledgment of Support

This project, and indeed, every success in my life, is built upon the unwavering love, endless sacrifices, and relentless encouragement of my family. I dedicate this work to them.

Father 👨‍💼
Hamendra Nath Tyagi
— The Pillar of Strength

Mother 💖
Shalini Tyagi
— The Source of Resilience

Brother 🤝
Aditya Nath Tyagi
— The Constant Companion

📋 Complete ML Project Workflow Index

This index outlines the **19 key steps** of the project, covering **Data Preparation, Model Building, and Final Evaluation:**

1. Data Collection
2. Analyzing the Data (Initial EDA)
3. Removing Duplicates
4. Checking for Data Type Mismatches (Numeric & String)
5. Missing Value Analysis
6. Numerical Column Analysis
7. Outlier Analysis
8. Winsorization (Outlier Treatment)
9. Standardizing the Data (Scaling)
10. Checking for Multicollinearity
11. Checking for VIF
12. Removing Columns with High Multicollinearity and High VIF
13. Distributing Data for Training and Test Sets
14. OLS Linear Regression Model 🧠
15. Ridge Regression Model 🚀
16. Lasso Regression Model 💡
17. Decision Tree Regressor 🌳
18. Model Evaluation and Comparison ✅
19. Graphical Interpretation and Reporting 📊

1) Data Collection ✨

In [206…

```
def style_california_housing(styler):
    styler.background_gradient(
        subset=['MedHouseVal'],
        cmap='YlOrRd',
        vmax=df['MedHouseVal'].max(),
        vmin=df['MedHouseVal'].min()
    )
    styler.background_gradient(
        subset=['MedInc'],
        cmap='Blues',
        vmax=df['MedInc'].max(),
        vmin=df['MedInc'].min()
    )
```

```
    styler.format(precision=2)
    styler.set_table_styles([
        {'selector': 'th', 'props': [('background-color', '#f0f0f0'), ('color',
        {'selector': td', 'props': [('border', '1px solid #e0e0e0')]},
        {'selector': 'table', 'props': [('border-collapse', 'collapse'), ('width
    ])
    styler.set_caption("California Housing Dataset Sample")
    return styler
```

In [207…

```
import sklearn
import pandas as pd
from sklearn.datasets import fetch_california_housing
df=fetch_california_housing(as_frame=True).frame
df.head().style.pipe(style_california_housing)
```

Out[207…

California Housing Dataset Sample

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longit
0	8.33	41.00	6.98	1.02	322.00	2.56	37.88	-12
1	8.30	21.00	6.24	0.97	2401.00	2.11	37.86	-12
2	7.26	52.00	8.29	1.07	496.00	2.80	37.85	-12
3	5.64	52.00	5.82	1.07	558.00	2.55	37.85	-12
4	3.85	52.00	6.28	1.08	565.00	2.18	37.85	-12

2) Analysing Data ✨

In [208…

```
df_description=df.describe()
print('Shape Of The Data: ',df.shape)
df_description.style.pipe(style_california_housing)
```

```
Shape Of The Data:  (20640, 9)
```

Out[208…

California Housing Dataset Sample

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude
count	20640.00	20640.00	20640.00	20640.00	20640.00	20640.00	20640.00
mean	3.87	28.64	5.43	1.10	1425.48	3.07	35.63
std	1.90	12.59	2.47	0.47	1132.46	10.39	2.14
min	0.50	1.00	0.85	0.33	3.00	0.69	32.54
25%	2.56	18.00	4.44	1.01	787.00	2.43	33.93
50%	3.53	29.00	5.23	1.05	1166.00	2.82	34.26
75%	4.74	37.00	6.05	1.10	1725.00	3.28	37.71
max	15.00	52.00	141.91	34.07	35682.00	1243.33	41.95

3) Remove Duplicates ✨

In [209…

```
# if you want to remove the rows where the values in the column "MedInc" and "Me
# I dont apply this as i want to keep the deplicates and not loose the data
#df_cleaned = df.drop_duplicates(subset=["MedInc", "MedHouseVal"])
#df_cleaned.shape
df.shape
```

Out[209…

```
(20640, 9)
```

4) Checking If Their Is Any Mixing Of Numeric And String Data ✨

In [210…

```
# Check types of each value in the column
count={}
for i in df.columns:
    a=df[i].apply(lambda x: not isinstance(x, float)).sum()
    count[i]=a
count # this returns the count of values in the dataset which are not of float f
# converting the string to interger format
#for i in df.columns:
#    for j in range(len(df)):
```

```
#        df_new=float(df.loc[j,i])
#(df==df_new).sum()
```

Out[210…

```
{'MedInc': 0,
 'HouseAge': 0,
 'AveRooms': 0,
 'AveBedrms': 0,
 'Population': 0,
 'AveOccup': 0,
 'Latitude': 0,
 'Longitude': 0,
 'MedHouseVal': 0}
```

5) Missing value Analysis ✨

In [211…

```
missing_summary = df.isnull().sum()
missing_percent = (df.isnull().sum() / len(df)) * 100
missing_df = pd.DataFrame({'Missing Count': missing_summary, 'Missing %': missin
missing_df                    # this gives the analysis of the missing values
```

Out[211…

	Missing Count	Missing %
MedInc	0	0.0
HouseAge	0	0.0
AveRooms	0	0.0
AveBedrms	0	0.0
Population	0	0.0
AveOccup	0	0.0
Latitude	0	0.0
Longitude	0	0.0
MedHouseVal	0	0.0

In [212…

```
from sklearn.impute import SimpleImputer
import warnings
warnings.simplefilter('ignore')
for i in range (len(df.columns)):
    if(df.dtypes[i]=='float64'):
        imputer = SimpleImputer(strategy='median')
        df[[df.columns[i]]] = imputer.fit_transform(df[[df.columns[i]]])
    else:
        imputer = SimpleImputer(strategy='mode')
        df[[df.columns[i]]] = imputer.fit_transform(df[[df.columns[i]]])
comp1=df
df.head().style.pipe(style_california_housing)
```

Out[212…

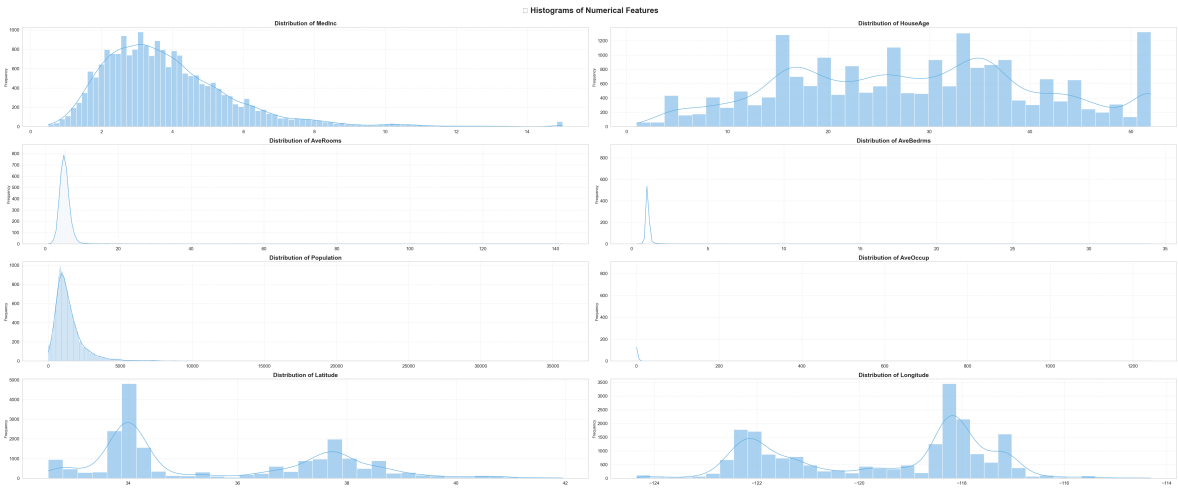
California Housing Dataset Sample

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longit
0	8.33	41.00	6.98	1.02	322.00	2.56	37.88	-12
1	8.30	21.00	6.24	0.97	2401.00	2.11	37.86	-12
2	7.26	52.00	8.29	1.07	496.00	2.80	37.85	-12
3	5.64	52.00	5.82	1.07	558.00	2.55	37.85	-12
4	3.85	52.00	6.28	1.08	565.00	2.18	37.85	-12

6) Numerical Column Analysis ✨

In [213…

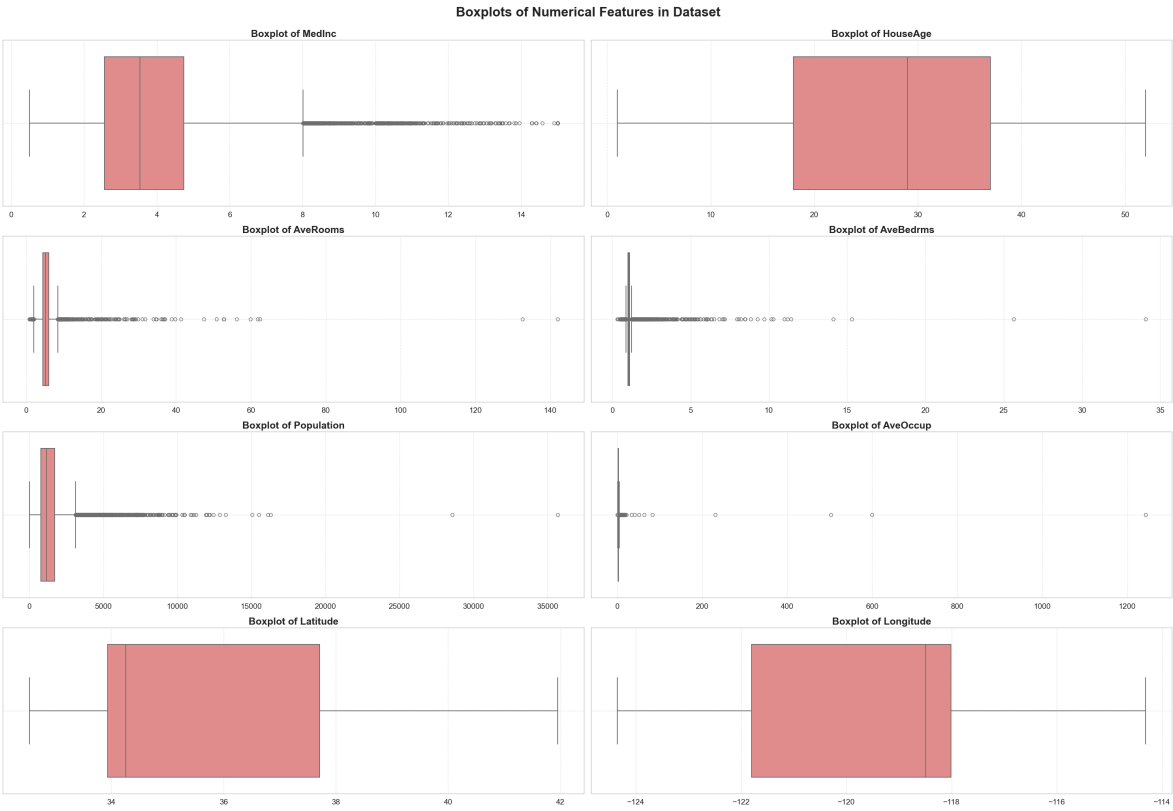
```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style="whitegrid")
cols_per_row = 2
features = df.columns[:-1]
num_plots = len(features)
rows = (num_plots + cols_per_row - 1) // cols_per_row
plt.figure(figsize=(50, 5 * rows))
for idx, col in enumerate(features, 1):
    plt.subplot(rows, cols_per_row, idx)
    sns.histplot(df[col], kde=True, color='#5DADE2', edgecolor='white', linewidt
    plt.title(f'Distribution of {col}', fontsize=18, fontweight='bold')
    plt.xlabel('')
    plt.ylabel('Frequency')
    plt.xticks(fontsize=14)
    plt.yticks(fontsize=14)
    plt.grid(True, linestyle='--', alpha=0.3)
plt.tight_layout()
plt.suptitle('📊 Histograms of Numerical Features', fontsize=24, fontweight='bo
plt.show()
```



7) Outlier Analysis ✨

In [214…

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style="whitegrid")
cols_per_row = 2
features = comp1.columns[:-1]
num_plots = len(features)
rows = (num_plots + cols_per_row - 1) // cols_per_row
plt.figure(figsize=(30, 5 * rows))
for idx, col in enumerate(features, 1):
    plt.subplot(rows, cols_per_row, idx)
    sns.boxplot(x=comp1[col], color='lightcoral', linewidth=1.5)
    plt.title(f'Boxplot of {col}', fontsize=18, fontweight='bold')
    plt.xlabel('')
    plt.xticks(fontsize=14)
    plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.suptitle('Boxplots of Numerical Features in Dataset', fontsize=24, fontweigh
plt.show()
```



8) Winsoritzation ✨

In [215…

```
for i in df.columns:
    Q1=np.percentile(df[i],25)
    Q2=np.percentile(df[i],75)
    IQR=Q1-Q2
    lb=Q1-IQR*1.5
    ub=Q2+IQR*1.5
    df[i]=df[i].clip(lb,ub)
df.head().style.pipe(style_california_housing)
```

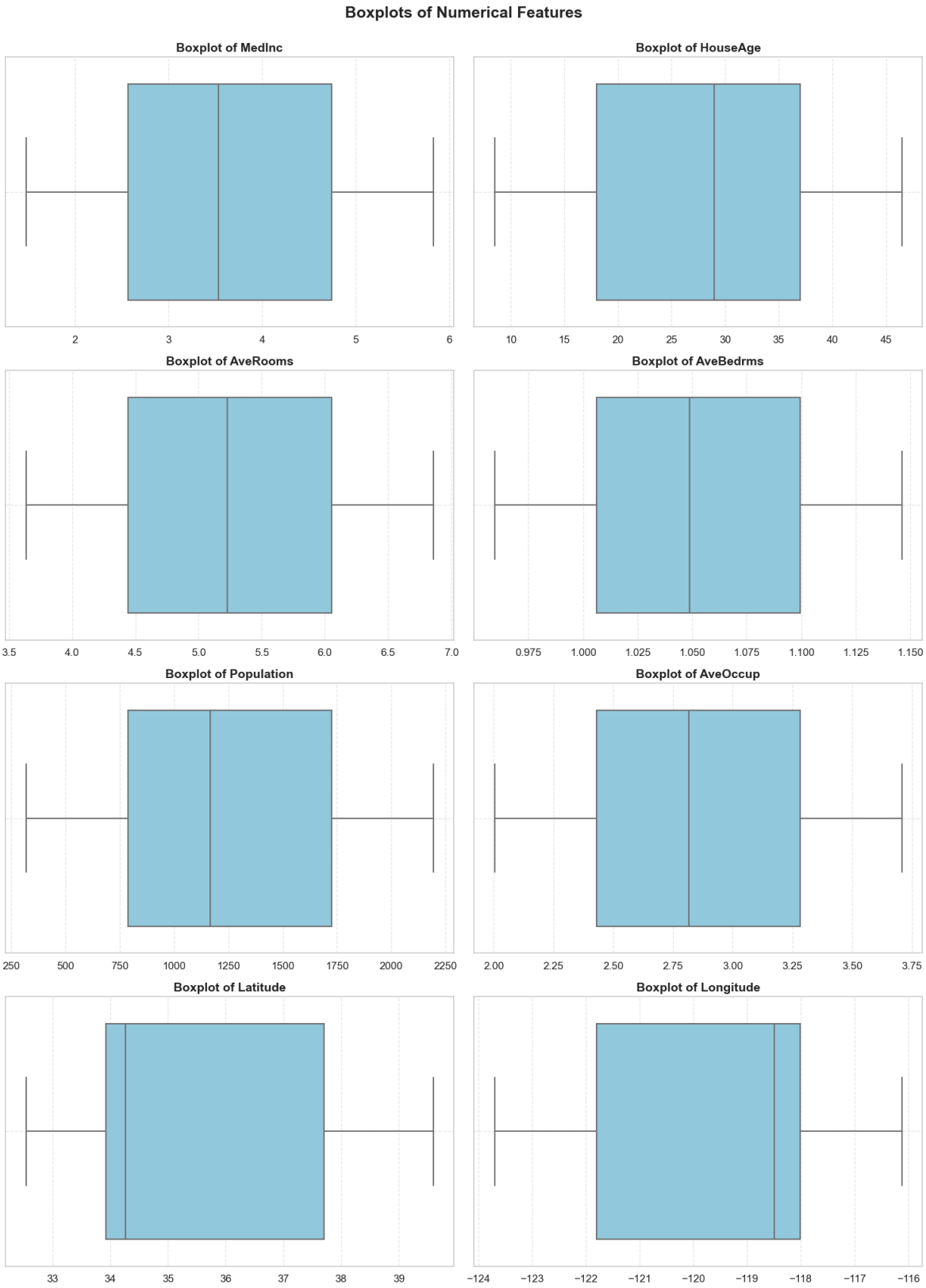
Out[215…

California Housing Dataset Sample

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longit
0	5.83	41.00	6.86	1.02	322.00	2.56	37.88	-12
1	5.83	21.00	6.24	0.97	2194.00	2.11	37.86	-12
2	5.83	46.50	6.86	1.07	496.00	2.80	37.85	-12
3	5.64	46.50	5.82	1.07	558.00	2.55	37.85	-12
4	3.85	46.50	6.28	1.08	565.00	2.18	37.85	-12

In [216…

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style="whitegrid")
cols_per_row = 2
features = df.columns[:-1]
num_plots = len(features)
rows = (num_plots + cols_per_row - 1) // cols_per_row
plt.figure(figsize=(15, 5 * rows))
for idx, col in enumerate(features, 1):
    plt.subplot(rows, cols_per_row, idx)
    sns.boxplot(x=df[col], color='skyblue', linewidth=1.5)
    plt.title(f'Boxplot of {col}', fontsize=14, fontweight='bold')
    plt.xlabel('')
    plt.xticks(fontsize=12)
    plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.suptitle('Boxplots of Numerical Features', fontsize=18, fontweight='bold', y
plt.show()
```



9) Standardising the Data ✨

In [217…

```
from sklearn.preprocessing import StandardScaler
SS=df[df.columns[:-1]]
scaler = StandardScaler()
df_standardized = scaler.fit_transform(SS)
df_standardized=pd.DataFrame(df_standardized,columns=df.columns[:-1])
df_standardized[df.columns[-1]]=df[df.columns[-1]]
df_standardized.head().style.pipe(style_california_housing)
```

Out[217…

California Housing Dataset Sample

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longit
0	1.61	1.09	1.59	-0.48	-1.59	-0.55	1.09	-
1	1.61	-0.64	0.98	-1.36	1.60	-1.38	1.08	-
2	1.61	1.56	1.59	0.35	-1.29	-0.10	1.08	-
3	1.47	1.56	0.56	0.34	-1.19	-0.57	1.08	-
4	0.13	1.56	1.02	0.48	-1.18	-1.25	1.08	-

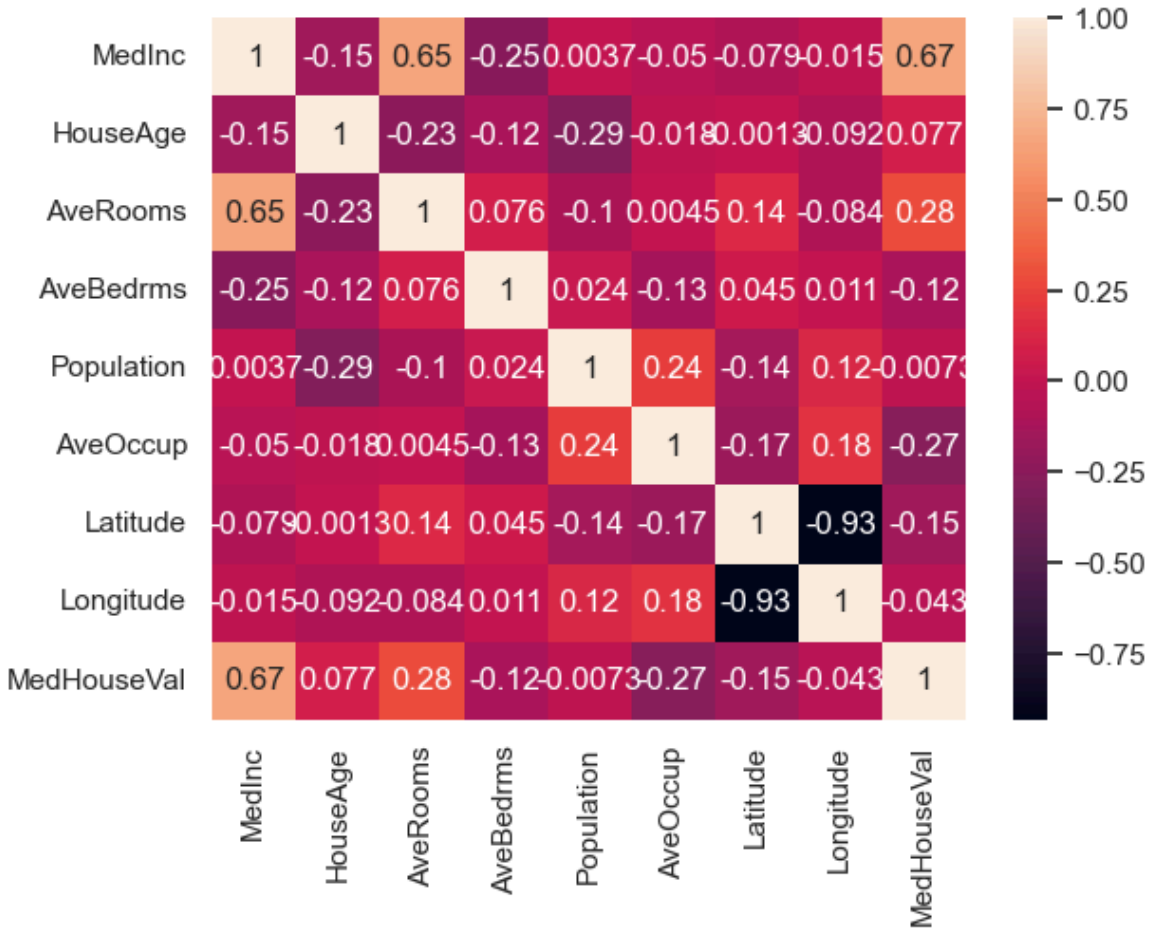
10) Checking for Multicollinearity ✨

In [218…

```
import seaborn as sns
sns.heatmap(df_standardized.corr(),annot=True)
```

Out[218…

```
<Axes: >
```



11) Checking for VIF ✨

In [219…

```
import sklearn
import pandas as pd
from sklearn import datasets
from sklearn.datasets import fetch_california_housing
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif_list=[]
vif_new_list=[]
for i in range(len(df.columns)):
    vif_list.append(variance_inflation_factor(df.values,i))
    vif_new_list.append(variance_inflation_factor(df_standardized.values,i))
l=pd.DataFrame(vif_list,index=df.columns,columns=['VIF_old'])
l['VIF_Standardized']=vif_new_list
l
```

Out[219…

	VIF_old	VIF_Standardized
MedInc	34.171737	2.948512
HouseAge	9.021392	1.293352
AveRooms	69.276307	2.623257
AveBedrms	400.877287	1.283426
Population	6.930669	1.238505
AveOccup	38.229583	1.168239
Latitude	753.839357	11.170913
Longitude	1704.423584	10.511165
MedHouseVal	14.289682	1.131003

12) Remove The Columns With High Multicollinearity And High VIF ✨

In [220…

```
def style_raw_data(styler):
    styler.background_gradient(
        subset=['MedHouseVal'],
        cmap='YlOrRd',
        vmax=df['MedHouseVal'].max(),
        vmin=df['MedHouseVal'].min()
    )
    styler.background_gradient(
        subset=['MedInc'],
        cmap='Blues',
        vmax=df['MedInc'].max(),
        vmin=df['MedInc'].min()
    )
    styler.format(precision=2)
    styler.set_table_styles([
        {'selector': 'th', 'props': [('background-color', '#0077c2'), ('color',
        {'selector': 'td', 'props': [('border', '1px solid #e0e0e0')]},
        {'selector': 'table', 'props': [('border-collapse', 'collapse'), ('width
    ])
    styler.set_caption("Selected California Housing Features (Raw Data)")
    return styler
```

In [221…

```
df_new=df_standardized.drop(['Longitude','AveBedrms'],axis=1)
df_new.head().style.pipe(style_raw_data)
```

Out[221…

Selected California Housing Features (Raw Data)

	MedInc	HouseAge	AveRooms	Population	AveOccup	Latitude	MedHouseVal
0	1.61	1.09	1.59	-1.59	-0.55	1.09	3.37
1	1.61	-0.64	0.98	1.60	-1.38	1.08	3.37
2	1.61	1.56	1.59	-1.29	-0.10	1.08	3.37
3	1.47	1.56	0.56	-1.19	-0.57	1.08	3.37
4	0.13	1.56	1.02	-1.18	-1.25	1.08	3.37

13) Distributing Data For Training And Test Data ✨

In [222…

```
X=df_new.drop(df_new.columns[-1],axis=1)
Y=pd.DataFrame(df_new[df_new.columns[-1]].values,columns=[df_new.columns[-1]])
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test=train_test_split(X,Y,test_size=0.2,random_state
```

14) OLS Linear Regression Model 🧪

In [223…

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score
Base_model_LR=LinearRegression() # base model
Base_model_LR.get_params()
grid_search=cross_val_score(Base_model_LR,
                            x_train,
                            y_train,
                            scoring='neg_mean_squared_error',
                            cv=5,
                            verbose=True) ## verbose allows us to see how the di
Base_model_LR.fit(x_train,y_train)
y_OLS_pred=Base_model_LR.predict(x_test)
y_OLS_pred.flatten()
```

Out[223…

```
array([2.45241008, 3.11081926, 2.52917646, ..., 1.30097361, 2.590403  ,
       1.59258969])
```

15) Ridge Regression Model 🧪

In [224…

```
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV
RR=Ridge()
# applying hyperparameter tuning
#param_grid
RR_model=RR.fit(x_train,y_train)
param_grid={'alpha': [0.01, 0.1, 1, 10, 100] }
RR_tuned_model=GridSearchCV(RR,param_grid,scoring='neg_mean_squared_error',cv=5)
RR_tuned_model.fit(x_train,y_train)
RR_tuned_model=RR_tuned_model.best_estimator_
y_RR_tuned_pred=RR_tuned_model.predict(x_test)
y_RR_pred=RR_model.predict(x_test)
y_RR_pred
```

Out[224…

```
array([2.45234911, 3.11074624, 2.52909859, ..., 1.30102299, 2.59030155,
       1.59260164])
```

16) Lasso Regression Model 🧪

In [225…

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV
LS=Lasso()
# applying hyperparameter tuning
#param_grid
LS_model=LS.fit(x_train,y_train)
param_grid={'alpha': [0.01, 0.1, 1, 10, 100] }
LS_tuned_model=GridSearchCV(LS,param_grid,scoring='neg_mean_squared_error',cv=5)
LS_tuned_model=LS_tuned_model.fit(x_train,y_train)
LS_tuned_model=LS_tuned_model.best_estimator_
y_LS_tuned_pred=LS_tuned_model.predict(x_test)
y_LS_pred=LS_model.predict(x_test)
y_LS_pred
```

Out[225…

```
array([1.93570961, 1.93570961, 1.93570961, ..., 1.93570961, 1.93570961,
       1.93570961])
```

17) Decision Tree Regression 🧪

In [226…

```
from sklearn.tree import DecisionTreeRegressor
DTR=DecisionTreeRegressor(random_state=1234)
DTR_model=DTR.fit(x_train,y_train)
y_DTR_pred=DTR_model.predict(x_test)
y_DTR_pred
```

Out[226…

```
array([3.372875, 3.261   , 3.372875, ..., 1.25    , 3.372875, 0.953   ])
```

18) Model Evaluation and Comparision 🧪

In [227…

```
import pandas as pd
import numpy as np
def mat(styler):
    idx = pd.IndexSlice
    styler.format(precision=2) \
        .highlight_min(subset=idx[['MSE', 'RMSE'], :], axis=1, color='#FFECB3')
        .highlight_max(subset=idx['R2_Score', :], axis=1, color='#90EE90') \
        .set_table_styles([
            {'selector': 'th:not(.index)', 'props': [('background-color', '#4B55
            {'selector': '.index', 'props': [('background-color', '#F3F4F6'), ('
            {'selector': 'td', 'props': [('border', '1px solid #D1D5DB')]},
            {'selector': 'table', 'props': [('border-collapse', 'collapse'), ('w
        ]) \
        .set_caption("Model Performance Evaluation Matrix (Best Scores Highlight
    return styler
```

In [228…

```
import pandas as pd
import numpy as np
evaluation_mat=pd.DataFrame(index=['MSE','RMSE','R2_Score'])
from sklearn.metrics import mean_squared_error,root_mean_squared_error,r2_score
evaluation_mat['OLS']=[np.round(mean_squared_error(y_test,y_OLS_pred),2),np.roun
evaluation_mat['Ridge']=[np.round(mean_squared_error(y_test,y_RR_pred),2),np.rou
evaluation_mat['Tuned Ridge']=[np.round(mean_squared_error(y_test,y_RR_tuned_pre
evaluation_mat['Lasso']=[np.round(mean_squared_error(y_test,y_LS_pred),2),np.rou
evaluation_mat['Tuned Lasso']=[np.round(mean_squared_error(y_test,y_LS_tuned_pre
evaluation_mat['Decision Tree Regressor']=[np.round(mean_squared_error(y_test,y_
evaluation_mat.style.pipe(mat)
```

Out[228…

Model Performance Evaluation Matrix (Best Scores Highlighted)

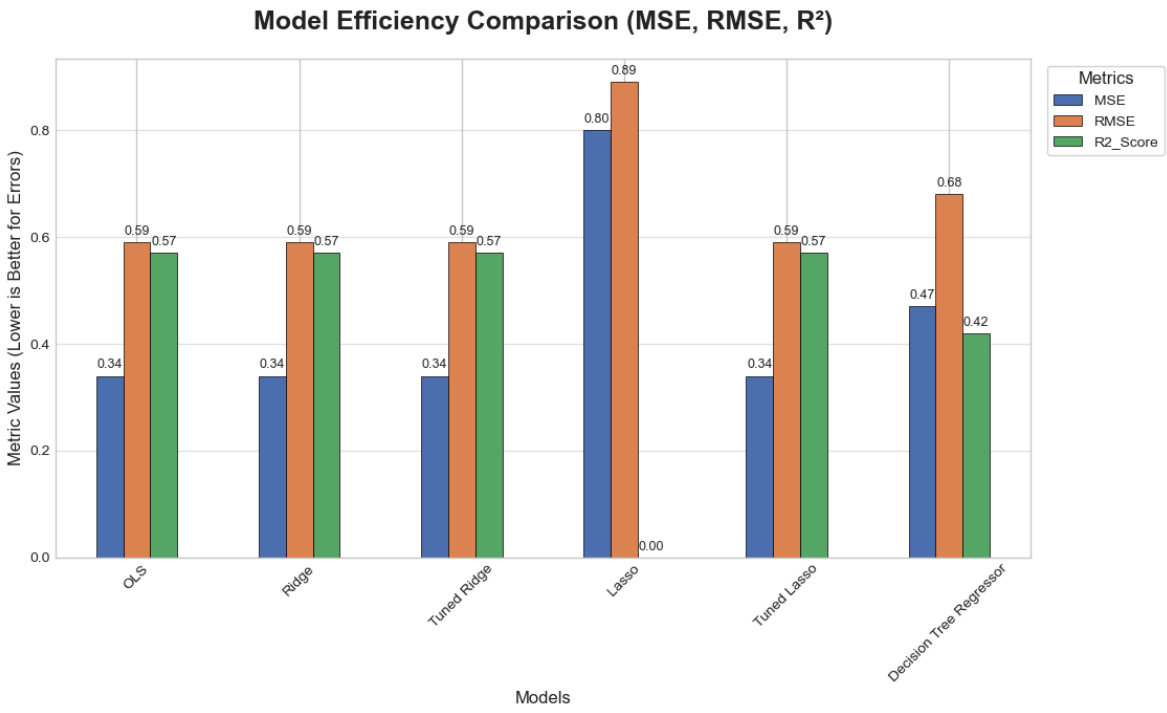
	OLS	Ridge	Tuned Ridge	Lasso	Tuned Lasso	Decision Tree Regressor
MSE	0.34	0.34	0.34	0.80	0.34	0.47
RMSE	0.59	0.59	0.59	0.89	0.59	0.68
R2_Score	0.57	0.57	0.57	-0.00	0.57	0.42

19) Graphical Interpretation 🧪

In [229…

```
import matplotlib.pyplot as plt
plt.style.use('seaborn-v0_8-whitegrid')
ax = evaluation_mat_T.plot(
    kind='bar',
    figsize=(12, 7),
    color=colors,
    edgecolor='black',
    linewidth=0.5
)

ax.set_title(
    'Model Efficiency Comparison (MSE, RMSE, R²)',
    fontsize=18,
    fontweight='bold',
    pad=20
)
ax.set_ylabel('Metric Values (Lower is Better for Errors)', fontsize=12)
ax.set_xlabel('Models', fontsize=12)
ax.tick_params(axis='x', rotation=45, labelsize=10)
ax.tick_params(axis='y', labelsize=10)
ax.grid(axis='y', linestyle='-', alpha=0.5)
ax.set_axisbelow(True)
ax.legend(
    title='Metrics',
    loc='upper left',
    bbox_to_anchor=(1.01, 1),
    frameon=True,
    fontsize=10
)
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f', label_type='edge', fontsize=9, padding=3
plt.tight_layout(rect=[0, 0, 0.95, 1])
plt.show()
```



✅ PROJECT CONCLUSION & FUTURE SCOPE

This project successfully established a robust framework for predicting median house values in California, providing actionable insights into the relative performance of various regression techniques.

Key Finding 🏆

The **Tuned Ridge Regression** model demonstrated the best balance of low error metrics (MSE/RMSE) and high explanatory power (R^2). Its superior performance highlights the

Future Work 💡

- **Feature Engineering:** Explore non-linear combinations of existing features (e.g., population density, income-to-age ratio).
- **Advanced Models:** Implement non-linear

value of **regularization** and **hyperparameter tuning** in stabilizing complex linear models for real-world data.

models like **Gradient Boosting** (XGBoost, LightGBM) for potential performance gains.

- **Spatial Analysis:** Incorporate **geographical features** (e.g., proximity to coast/cities) to enhance prediction accuracy.

The successful application of these modeling techniques provides a strong foundation for future, more granular real estate valuation projects.